LINKED LIST MANAGEMENT SYSTEM

DSA Mini Project

Submitted By Naman Kumar

University: Sharda University

Semester: 3rd Semester

Year: 2nd Year

Language Used: Java

1. Introduction

A Linked List is a linear data structure in which elements are stored in nodes. Each node contains data and references that connect it to other nodes.

This project implements two types of linked lists using Java: Singly Linked List and Doubly Linked List. The project provides a menu-driven system to perform different operations on both linked lists.

2. Objective

The main objectives of this project are:
• To understand Linked List data structures.
• To implement a Singly Linked List using Java.
• To implement a Doubly Linked List using Java.
• To perform insertion and deletion operations.
• To perform searching and updating.
• To find maximum and minimum values.
• To find the middle element.
• To reverse and sort the list.
• To remove duplicate values.
• To understand the difference between Singly and Doubly Linked Lists.

3. Software and Hardware Requirements

Software Requirements
• Java JDK
• Visual Studio Code or Eclipse
• Windows/Linux/macOS

Hardware Requirements
• Computer or Laptop
• Minimum 4 GB RAM

4. Singly Linked List

A Singly Linked List consists of nodes where each node contains two parts: data and a next reference. The next reference stores the connection to the next node.

Example:

```
10 -> 20 -> 30 -> 40 -> NULL
```

5. Singly Linked List Operations

The implemented operations are:
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Insert After a Node
5. Insert Before a Node
6. Delete from Beginning
7. Delete from End

8. Delete from Position
9. Delete by Value
10. Search
11. Update
12. Display
13. Count Nodes
14. Find Maximum
15. Find Minimum
16. Find Middle Element
17. Reverse
18. Sort
19. Remove Duplicates

6. Doubly Linked List

A Doubly Linked List contains three parts in each node: previous reference, data, and next reference. The prev reference points to the previous node while the next reference points to the next node.

Example:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

7. Doubly Linked List Operations

The implemented operations are:
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Insert After a Node
5. Insert Before a Node
6. Delete from Beginning
7. Delete from End
8. Delete from Position
9. Delete by Value
10. Search
11. Update
12. Display Forward
13. Display Backward
14. Count Nodes
15. Find Maximum
16. Find Minimum
17. Find Middle Element
18. Reverse
19. Sort
20. Remove Duplicates

8. Difference Between Singly and Doubly Linked List

Singly Linked List uses data and a next reference. It supports forward traversal and uses less memory. Doubly Linked List uses data, prev, and next references, allowing forward and backward traversal but

requiring additional memory.

Singly Linked List	Doubly Linked List
Data + next reference	Data + prev + next references
Forward traversal	Forward and backward traversal
Less memory	More memory
Simpler implementation	More link management

9. Time Complexity

The typical complexity of major operations is shown below.

Operation	Time Complexity
Insert at Beginning	O(1)
Insert at End	O(n)
Insert at Position	O(n)
Delete from Beginning	O(1)
Delete from End	O(n)
Search / Display / Count	O(n)
Find Maximum / Minimum	O(n)
Find Middle	O(n)
Reverse	O(n)
Bubble Sort	$O(n^2)$
Remove Duplicates	$O(n^2)$

10. Testing and Edge Cases

Important test cases include empty linked list, one-node list, insertion/deletion at position 1, last valid position, invalid or out-of-range position, deleting a value that does not exist, duplicate values, zero and negative values, sorting an empty or single-node list, and reversing an empty or single-node list.

11. Sample Output

The following demonstrates the expected style of output for the project.

```
Singly Linked List

Before:
40 -> 10 -> 30 -> 20 -> 50 -> NULL

After Sort:
10 -> 20 -> 30 -> 40 -> 50 -> NULL

After Reverse:
```

```
50 -> 40 -> 30 -> 20 -> 10 -> NULL

Doubly Linked List

Forward:
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL

Backward:
NULL <- 40 <-> 30 <-> 20 <-> 10 -> NULL
```

12. Conclusion

The Linked List Management System provides a practical implementation of Singly and Doubly Linked Lists using Java. The project demonstrates insertion, deletion, searching, updating, counting, finding minimum and maximum values, finding the middle element, reversing, sorting, and removing duplicates. The project helps in understanding node creation, traversal, and link management in linked list data structures.

13. Author

Naman Kumar
Sharda University
Semester 3
Year 2